

M24: What's Next — The Agent Frontier

You have built agents that reason, retrieve, coordinate, and deploy. Now look ahead — at the emerging ecosystem, evolving capabilities, and your personal development path.

Learning Objectives

- ☐ Understand emerging agent-to-agent communication protocols and marketplace patterns
- ☐ Explore Claude's evolving capabilities — computer use, extended thinking, and expanded tool use
- ☐ Apply responsible AI principles to agent design, including human-in-the-loop and fail-safe defaults
- ☐ Identify communities, papers, and open-source frameworks for continued learning
- ☐ Create a personal 90-day agent development roadmap with concrete milestones

Emerging Patterns: Agent-to-Agent Protocols

Everyday Analogy

The early internet was a collection of isolated websites. You visited one page, got your information, and manually navigated to the next. There was no way for one website to automatically talk to another. Then APIs arrived, and suddenly every service could connect — your calendar could talk to your email, your store could talk to your shipping provider, your bank could talk to your payment processor.

The pain of the pre-API era was enormous: if you wanted to check flight status and hotel availability and rental car pricing for one trip, you had to visit three separate websites, copy data between browser tabs, and manually compare results. Integration was you, the human, doing copy-paste.

Agents are at that same inflection point right now. Today, most agents are standalone bots — they do their one thing well but cannot discover or collaborate with other agents. Agent-to-agent protocols like MCP are the "API moment" for AI: the one universal plug that lets any agent discover, negotiate with, and delegate to any other agent.

By 2025, this is no longer hypothetical. Google's **A2A specification** (announced April 2025, governed under the Linux Foundation) defines the wire format that lets independently built agents discover each other and exchange work. The canonical artifact in A2A is the **Agent Card** — a JSON document published at a well-known URL (`/ .well-known/agent . json`) that describes one agent's identity, capabilities, and authentication requirements:

```
{
  "name": "tax-specialist",
  "description": "US federal & state tax analysis with deduction optimization.",
  "url": "https://agents.acme.com/tax-specialist",
  "version": "2.1.0",
  "provider": { "organization": "Acme Tax LLC" },
  "capabilities": {
    "streaming": true,
    "pushNotifications": true,
    "stateTransitionHistory": true
  },
  "authentication": { "schemes": ["bearer", "oauth2"] },
  "defaultInputModes": ["text", "application/json"],
  "defaultOutputModes": ["text", "application/json"],
  "skills": [
    {
      "id": "deduction-analysis",
      "name": "Analyze itemized deductions",
      "description": "Given income + receipts, returns optimal deduction plan.",
      "tags": ["tax", "deductions", "us-federal"],
      "examples": ["Analyze deductions for $85K income, married filing jointly"]
    }
  ]
}
```

An orchestrator agent fetches this card, sees that the tax-specialist supports streaming and OAuth, and knows exactly which skill to invoke. The same orchestrator can call ANY A2A-compliant specialist — the wire format is identical whether the specialist was built with the Claude Agent SDK, LangGraph, CrewAI, or someone's bespoke Go service.

Technical Definition — Two Protocols, Two Layers

By 2026 the agent-interop landscape has crystallized into **two complementary protocols**. Confusing them is the #1 mistake when people first talk about “agent protocols.”

1. MCP — agent ↔ tool. MCP (you learned this in M07) is how a single agent connects to its tools: filesystem, GitHub, Postgres, an internal Jira API. The agent is the client; the tool exposes itself as an MCP server. One agent, many tools.

2. A2A — agent ↔ agent. A2A is how one agent talks to another agent. Each A2A agent publishes an **Agent Card** (the JSON above), exposes a small HTTP surface (`tasks/send`, `tasks/get`, `tasks/cancel`, plus an SSE stream), and treats incoming work as **Tasks** with a defined lifecycle: `submitted` → `working` → `input-required` → `completed` (or `failed` / `canceled`).

The stack: a production multi-agent system uses BOTH. An orchestrator agent uses A2A to discover and delegate to a tax-specialist agent. That tax-specialist, internally, uses MCP to talk to its accounting database, file storage, and IRS lookup API. Two protocols, two scopes — not competitors.

Decision Guide — MCP vs A2A

Scenario	Reach for	Why
Expose a Postgres database to your agent	MCP	Tool integration, no agent on the other side
Your travel agent needs to call a vendor's flight-pricing agent	A2A	Cross-org, agent-to-agent delegation with auth
In-process subagent (M14) inside one process	Neither	A function call is fine; protocols add overhead
Pluggable specialist marketplace inside your enterprise	A2A	Each team owns one agent; the registry is Agent Cards

AGENT-TO-AGENT PROTOCOL HANDSHAKE

Orchestrator

coordinator

① GET /.well-known/agent.json → *discover capabilities*

② CARD { skills: [deduction-analysis, ...] }

③ POST tasks/send { input: "\$85K income" }

④ task: completed { savings: \$2,728 }

↑ routed via **Agent Marketplace** (discovery broker)

Tax Specialist

domain expert

Why It Matters

Today, connecting Claude to a new data source requires custom integration code. With standardized protocols, adding a new capability becomes as simple as connecting to a published MCP server — no custom code required. Companies building agents today report spending 40-60% of development time on integrations. Agent protocols could reduce that to near zero, the same way REST APIs eliminated custom wire protocols in web development.

⚠ Common Misconceptions

“MCP and A2A are competitors.” — They aren't. MCP = *agent ↔ tool*; A2A = *agent ↔ agent*. Real systems use both: A2A on the outside, MCP on the inside.

“Agents will negotiate in natural language.” — No. A2A is JSON over HTTP(S) with SSE / webhooks. Natural language goes *inside* a message's text part, not on the wire.

“Same-process subagents need a protocol.” — They don't. M14 subagents are function calls. Reach for A2A only across process / org / trust boundaries.

“The Agent Card replaces an API contract.” — It's the *discovery* contract, not the data contract. Each skill still needs JSON Schema for its input/output shape.

Knowing the wire format is only half the story. Next, see what publishing and consuming an Agent Card looks like in real code — and where AGNTCY and ANP fit in the wider landscape.

A2A In Practice — Publish & Consume

Everyday Analogy

Posting an Agent Card is like printing business cards. The card itself is cheap and tells anyone who picks it up exactly what you do, how to reach you, and what authentication you accept. Calling another agent is like dialing the number on someone else's card — you have to handle the dial tone, the busy signal, and the ringback.

The pain of skipping this step is that every cross-team agent integration becomes a snowflake. One team uses gRPC, another uses HTTP with a custom auth scheme, a third uses a message bus — and your orchestrator agent grows a different client per vendor. A2A collapses that to one client: speak the protocol, fetch the card, post the task.

The mental model: **publish** = expose `/well-known/agent.json` + a small HTTP surface; **consume** = fetch card, post task, poll or stream for completion. Two halves of one protocol.

Publishing an Agent Card (Server Side)

This is what your tax-specialist agent runs. The framework details vary — here it's FastAPI on Python and Fastify on Node — but the surface area is identical: a static card endpoint plus a task lifecycle. The internal “do the work” function (`run_claude_tax_skill`) is just your existing Claude agent loop wrapped in an A2A envelope.

```

import uuid
from fastapi import FastAPI, HTTPException, Header
from pydantic import BaseModel

app = FastAPI()
TASKS: dict[str, dict] = {} # in-memory; real deployments use Redis/Postgres

AGENT_CARD = {
    "name": "tax-specialist",
    "description": "US federal & state tax analysis with deduction optimization.",
    "url": "https://agents.acme.com/tax-specialist",
    "version": "2.1.0",
    "capabilities": {"streaming": False, "pushNotifications": True},
    "authentication": {"schemes": ["bearer"]},
    "defaultInputModes": ["text", "application/json"],
    "defaultOutputModes": ["text", "application/json"],
    "skills": [{
        "id": "deduction-analysis",
        "name": "Analyze itemized deductions",
        "description": "Given income + receipts, returns optimal deduction plan.",
        "tags": ["tax", "deductions", "us-federal"],
    }],
}

@app.get("/.well-known/agent.json")
def get_card():
    return AGENT_CARD

class TaskSend(BaseModel):
    id: str | None = None
    message: dict # {"role": "user", "parts": [{"type": "text", "text": "..."}]}
    skillId: str | None = None

@app.post("/tasks/send")
async def send_task(req: TaskSend, authorization: str | None = Header(None)):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "Bearer token required")

    task_id = req.id or str(uuid.uuid4())
    TASKS[task_id] = {"id": task_id, "state": "working", "messages": [req.message]}
    try:
        # YOUR existing Claude agent loop – M05/M12/M15B style.
        result = run_claude_tax_skill(req.message, skill_id=req.skillId)
        TASKS[task_id].update({
            "state": "completed",
            "artifacts": [{"name": "report", "parts": [{"type": "text", "text": result}]}],
        })
    except Exception as e:
        TASKS[task_id].update({"state": "failed", "error": str(e)})
    return TASKS[task_id]

```

```

@app.get("/tasks/{task_id}")
def get_task(task_id: str):
    if task_id not in TASKS:
        raise HTTPException(404, "Unknown task")
    return TASKS[task_id]

def run_claude_tax_skill(message: dict, skill_id: str | None) -> str:
    # Hand off to your existing Claude agent – this is the only line you write
    # that's domain-specific. Everything around it is boilerplate A2A.
    return "Estimated savings: $2,728 (itemize, claim home-office + 401k)"

import Fastify from "fastify";
import { randomUUID } from "node:crypto";

const app = Fastify();
const TASKS = new Map<string, any>();

const AGENT_CARD = {
  name: "tax-specialist",
  description: "US federal & state tax analysis with deduction optimization.",
  url: "https://agents.acme.com/tax-specialist",
  version: "2.1.0",
  capabilities: { streaming: false, pushNotifications: true },
  authentication: { schemes: ["bearer"] },
  defaultInputModes: ["text", "application/json"],
  defaultOutputModes: ["text", "application/json"],
  skills: [{
    id: "deduction-analysis",
    name: "Analyze itemized deductions",
    description: "Given income + receipts, returns optimal deduction plan.",
    tags: ["tax", "deductions", "us-federal"],
  }],
};

app.get("/.well-known/agent.json", async () => AGENT_CARD);

app.post("/tasks/send", async (req, reply) => {
  const auth = req.headers.authorization;
  if (!auth?.startsWith("Bearer ")) return reply.code(401).send({ error: "Bearer token required" });

  const body = req.body as { id?: string; message: any; skillId?: string };
  const taskId = body.id ?? randomUUID();
  TASKS.set(taskId, { id: taskId, state: "working", messages: [body.message] });

  try {
    const result = await runClaudeTaxSkill(body.message, body.skillId);
    TASKS.set(taskId, {
      ...TASKS.get(taskId),
      state: "completed",
      artifacts: [{ name: "report", parts: [{ type: "text", text: result }] }],
    });
  } catch (e: any) {

```

```

    TASKS.set(taskId, { ...TASKS.get(taskId), state: "failed", error: e.message });
  }
  return TASKS.get(taskId);
});

app.get("/tasks/:id", async (req, reply) => {
  const t = TASKS.get((req.params as any).id);
  return t ?? reply.code(404).send({ error: "Unknown task" });
});

async function runClaudeTaxSkill(message: any, skillId?: string): Promise<string> {
  // Your existing Claude agent loop – same as the M15B style.
  return "Estimated savings: $2,728 (itemize, claim home-office + 401k)";
}

app.listen({ port: 8080 });

```

What Just Happened?

Agent Card

tasks/send

tasks/{id}

Consuming Another Agent (Client Side)

This is what the orchestrator runs when it decides to delegate. The agent-locator URL would normally come from a registry or directory; here we pass it directly.

```

import httpx, time, uuid

BASE = "https://agents.acme.com/tax-specialist"
TOKEN = "your-bearer-token"

def delegate_to_agent(question: str, *, timeout_s: int = 30) -> str:
    # 1. Discover – fetch the Agent Card and verify the skill exists.
    card = httpx.get(f"{BASE}/.well-known/agent.json", timeout=5).raise_for_status().json()
    skill = next((s for s in card["skills"] if s["id"] == "deduction-analysis"), None)
    if skill is None:
        raise RuntimeError(f"Agent {card['name']} doesn't advertise deduction-analysis")
    if "bearer" not in card["authentication"]["schemes"]:
        raise RuntimeError("Agent doesn't accept bearer auth – refuse to call")

    # 2. Send the task.
    task_id = str(uuid.uuid4())
    res = httpx.post(
        f"{BASE}/tasks/send",
        json={"id": task_id, "skillId": skill["id"],
            "message": {"role": "user", "parts": [{"type": "text", "text": question}]},
        headers={"Authorization": f"Bearer {TOKEN}"},
        timeout=10,
    ).raise_for_status().json()

    # 3. Poll until terminal state (or timeout). Real code uses SSE if the
    #     card advertises capabilities.streaming.
    deadline = time.monotonic() + timeout_s
    while res["state"] in ("submitted", "working", "input-required"):
        if time.monotonic() > deadline:
            httpx.post(f"{BASE}/tasks/{task_id}/cancel",
                headers={"Authorization": f"Bearer {TOKEN}"})
            raise TimeoutError(f"Task {task_id} exceeded {timeout_s}s")
        time.sleep(0.5)
        res = httpx.get(f"{BASE}/tasks/{task_id}",
            headers={"Authorization": f"Bearer {TOKEN}"}).json()

    if res["state"] != "completed":
        raise RuntimeError(f"Task {task_id} {res['state']}: {res.get('error')}")
    return res["artifacts"][0]["parts"][0]["text"]

```

```

const BASE = "https://agents.acme.com/tax-specialist";
const TOKEN = "your-bearer-token";

export async function delegateToAgent(question: string, timeoutMs = 30_000): Promise<string> {
    // 1. Discover – fetch the Agent Card and verify the skill exists.
    const cardRes = await fetch(`${BASE}/.well-known/agent.json`);
    if (!cardRes.ok) throw new Error(`Card fetch failed: ${cardRes.status}`);
    const card = await cardRes.json();
    const skill = card.skills.find((s: any) => s.id === "deduction-analysis");
    if (!skill) throw new Error(`${card.name} doesn't advertise deduction-analysis`);
    if (!card.authentication.schemes.includes("bearer")) {

```



```

    throw new Error("Agent doesn't accept bearer auth – refuse to call");
  }

  // 2. Send the task.
  const taskId = crypto.randomUUID();
  const sendRes = await fetch(`${BASE}/tasks/send`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "Authorization": `Bearer ${TOKEN}` },
    body: JSON.stringify({
      id: taskId, skillId: skill.id,
      message: { role: "user", parts: [{ type: "text", text: question }] },
    }),
  });
  if (!sendRes.ok) throw new Error(`Task send failed: ${sendRes.status}`);
  let task = await sendRes.json();

  // 3. Poll until terminal state.
  const deadline = Date.now() + timeoutMs;
  while ([ "submitted", "working", "input-required" ].includes(task.state)) {
    if (Date.now() > deadline) {
      await fetch(`${BASE}/tasks/${taskId}/cancel`,
        { method: "POST", headers: { Authorization: `Bearer ${TOKEN}` } });
      throw new Error(`Task ${taskId} exceeded ${timeoutMs}ms`);
    }
    await new Promise((r) => setTimeout(r, 500));
    const r = await fetch(`${BASE}/tasks/${taskId}`,
      { headers: { Authorization: `Bearer ${TOKEN}` } });
    task = await r.json();
  }

  if (task.state !== "completed") {
    throw new Error(`Task ${taskId} ${task.state}: ${task.error ?? "unknown"}`);
  }
  return task.artifacts[0].parts[0].text;
}

```

Production Notes (the parts the demo skips)

- **Streaming over polling:** if the card sets `capabilities.streaming: true`, swap the poll loop for SSE on `/tasks/{id}/events`.
- **Long tasks:** use `pushNotifications` webhooks instead of polling; handle `input-required` state for mid-task questions.
- **Auth:** bearer is the floor — production uses OAuth 2.0 with audience-restricted tokens.

A2A is the most-adopted spec, but it isn't the only one. Two more standards are worth knowing about so you can read between the lines when a vendor or framework references them.

Beyond A2A — AGNTCY, ANP, and the Wider Landscape

The Three Specs You'll See Referenced

1. **A2A** (Google → Linux Foundation, 2025). The widest adoption: Anthropic, Salesforce, SAP, Atlassian, MongoDB, JetBrains, and most of the agent-framework world ship A2A endpoints. This is the spec your code should target by default.
2. **AGNTCY** (Internet of Agents). Cisco-led, open-source. AGNTCY is broader than A2A: it covers identity (W3C DIDs for agents), a global agent directory, observability, and orchestration — not just the wire format. Think of AGNTCY as a *stack* with A2A-compatible interop at the wire layer.
3. **ANP** (Agent Network Protocol). A research-flavored, more decentralized take — DIDs for identity, peer-to-peer discovery, no required central registry. Less production adoption than A2A but worth tracking if your architecture needs to avoid any central directory.

Why It Matters

Practical answer: build to A2A today, watch AGNTCY for the broader stack story, and read ANP if you're designing for a fully decentralized scenario. The good news: all three converge on the same shape — agents have cards, tasks have lifecycles, auth is mandatory. If your code follows the A2A surface above, porting to AGNTCY or ANP later is mostly a redirect — not a rewrite.

Agent protocols solve the communication problem between agents. But what can individual agents actually do? Claude's own capabilities are expanding rapidly — and each new capability unlocks entirely new categories of applications.

Claude's Evolving Capabilities

Everyday Analogy

Early smartphones could make calls and send texts. That was it. Then came cameras, and suddenly you had a camera in your pocket. Then GPS, and you had a navigator. Then app stores, and you had a platform. Each new capability did not just add a feature — it unlocked entire categories of applications that nobody had imagined.

The pain of limited capability is invisible until it disappears. Before smartphone cameras, you could not take a photo of a whiteboard, text it to your team, and collaborate remotely. You did not miss it because you could not imagine it. But once it existed, you could not go back.

Claude's capability expansion follows the same trajectory. Each new feature — computer use, extended thinking, larger context — is not just an incremental improvement. It unlocks agent applications that were previously impossible. The agents you build will ride each wave of new capabilities.

Here is what these capabilities look like as real code. Enabling extended thinking is a single parameter change — but it unlocks a fundamentally different reasoning mode. Just like adding a GPS chip to a phone meant one hardware change but an entirely new category of apps, this one parameter change turns Claude from a responder into a planner:

```
# Before: standard response
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Design a microservices architecture for..."}]
)

# After: extended thinking enabled
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={
        "type": "enabled",
        "budget_tokens": 10000 # Up to 10K tokens for reasoning
    },
    messages=[{"role": "user", "content": "Design a microservices architecture for..."}]
)

# response.content now includes thinking blocks you can inspect for debugging
```

```
// Before: standard response
const response = await client.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 1024,
  messages: [{ role: "user", content: "Design a microservices architecture for..." }]
});

// After: extended thinking enabled
const response = await client.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 16000,
  thinking: {
    type: "enabled",
    budget_tokens: 10000 // Up to 10K tokens for reasoning
  },
  messages: [{ role: "user", content: "Design a microservices architecture for..." }]
});

// response.content now includes thinking blocks you can inspect for debugging
```

What Just Happened?



Technical Definition

Claude's capability roadmap includes several features that fundamentally change what agents can do:

Computer Use: Claude can observe and interact with computer interfaces — clicking buttons, typing text, and navigating menus, just like a human sitting at a screen. Why is that transformative? Because it means agents can operate ANY software, not just software with APIs.

Think about legacy enterprise systems — the kind that have been running for 15 years with no API and no plans to add one. Before computer use, automating those systems meant expensive custom integrations or screen-scraping hacks. Now Claude can use the same GUI a human would. That one capability unlocks automation of entire workflow categories that were previously untouchable.

How does this differ from the tool use you learned in M05-M07? Tool use gives Claude access to functions you define. Those functions have structured inputs and outputs, predictable behavior, and fast execution. Computer use is fundamentally different. Claude receives a screenshot, decides where to click or what to type, then receives the next screenshot. It is slower, less predictable, and more expensive — but it works with any software that has a visual interface.

Here is the simplest way to think about it: tool use is Claude calling a function. Computer use is Claude sitting at your desk using your mouse and keyboard.

Extended Thinking: Claude can perform multi-step reasoning in a dedicated thinking phase before responding. This is NOT the same as chain-of-thought prompting, where you instruct Claude to "think step by step" in your prompt.

The difference matters. Chain-of-thought is a prompting trick — you ask the model to show its work, and it does, but using the same compute it would use anyway. Extended thinking is model-native: Claude allocates dedicated compute specifically for reasoning before it starts writing the response.

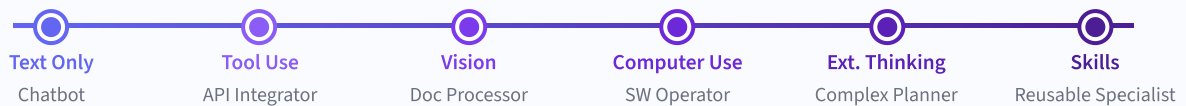
The result is dramatically better performance on complex tasks. Mathematical proofs, code architecture decisions, multi-constraint planning — all improve significantly. The thinking tokens are visible for debugging, so you can inspect Claude's reasoning process. They are billed separately from output tokens, so you can budget for reasoning independently.

Improved Tool Use: Each model generation brings more reliable structured output, better tool selection in ambiguous scenarios, and support for parallel tool calls. This means your agents become more reliable without you changing any code — just updating the model version.

Larger Context Windows: Expanding context means agents can process much more information in a single request. Instead of chunking a codebase into pieces and hoping the right chunk gets retrieved, you can pass the entire codebase in one go. The same applies to long regulatory documents. It also applies to extended conversation histories that would previously overflow the context.

For RAG applications (M09-M10), this shifts the tradeoff significantly. When the context window is large enough, you might not need RAG at all for some use cases — just paste the entire document into the prompt. That does not make RAG obsolete. Cost still matters: sending 200 pages per request is expensive. Latency still matters: larger prompts take longer to process. But for smaller document sets, you now have a simpler option.

CLAUDE CAPABILITY EVOLUTION TIMELINE



Skills — Capability Bundles

You have shipped agents in this course three ways: raw tools (M05), MCP servers (M07), and subagents (M14). **Skills** (Anthropic, late 2025) are a fourth way — and they sit between “a single tool” and “a whole subagent.” A Skill is a folder containing instructions, supporting files, and (optionally) executable scripts that the agent loads on demand when its task matches the skill’s description.

Everyday Analogy

A tool is a single kitchen utensil — a paring knife. An MCP server is the whole appliance — the oven, plumbed in once for any meal. A subagent is hiring a sous-chef. A Skill is a **recipe card**: short instructions plus references to ingredients and techniques, pulled from a binder only when tonight’s dish needs it.

The pain Skills solve: stuffing every domain rule into the system prompt makes the prompt huge, expensive to cache, and noisy for tasks that don’t need it. The pain MCP/subagents solve is different — *integration* and *parallelism*. Skills are about *just-in-time instructions*: the agent reads the skill folder only when it’s relevant.

Mental model: **tools** = verbs, **MCP** = wiring to systems, **subagents** = workers, **Skills** = on-demand playbooks. All four compose — a Skill can call tools, hit MCP servers, and spawn subagents.

A skill lives in a folder under `.claude/skills/` with a `SKILL.md` at the root. The frontmatter is what the agent sees during “skill discovery” — like an Agent Card for in-process capabilities. The body is the playbook itself, loaded only after the agent decides to use this skill:

```

---
name: pdf-extraction
description: Extract structured data (tables, key/value pairs, headings) from PDF
  files. Use this when the user uploads a PDF and asks for fields, columns,
  totals, or any structured content. Returns JSON.
allowed-tools: ["Read", "Bash"]
---

# PDF Extraction Skill

## When to use
- User uploads a PDF (.pdf in the conversation or in `./inputs/`).
- User asks for any structured field, table, or total.
- Source-of-truth answer needs to be grounded in the PDF, not Claude's guess.

## Procedure
1. Run `python scripts/extract.py path/to/file.pdf` (see reference/).
2. The script writes `out/extracted.json` with `pages[]`, `tables[]`, `kv[]`.
3. Validate against `reference/expected-schema.json`.
4. If validation fails, retry once with `--ocr` flag.
5. Return ONLY the requested fields – don't dump the whole JSON.

## Gotchas
- Scanned PDFs need `--ocr`. Detect: page text length < 50 chars => OCR.
- Multi-column layouts: pass `--layout=two-column`.
- Tables with merged cells: see `reference/merged-cells.md` for workaround.

```

Three things to notice. First, the **description** is the only part loaded into the agent's base prompt — it's how the agent decides whether the skill is relevant. Write it like a search-engine snippet, not like documentation. Second, **allowed-tools** scopes which tools this skill is permitted to use, even if the broader agent has more. Third, the body (the procedure, gotchas, referenced files) is only loaded when the skill activates — so the cost stays out of every turn.

Here's the agent-side surface when you use the Claude Agent SDK to register and invoke skills:

```

from claude_agent_sdk import ClaudeAgentClient, AgentOptions

# 1. Point the agent at a skills directory. The SDK walks the tree,
#   reads each SKILL.md frontmatter, and exposes the descriptions
#   to Claude as choosable capabilities.
agent = ClaudeAgentClient(
    options=AgentOptions(
        model="claude-opus-4-7",
        skill_dirs=["./.claude/skills"], # one or many folders
        allowed_tools=["Read", "Write", "Bash"],
    ),
)

# 2. Run a task – the agent will autonomously decide whether to
#   load `pdf-extraction` based on the user's request.
async for event in agent.run("Extract the totals table from invoice.pdf"):
    if event.type == "skill_loaded":
        print(f"→ loaded skill: {event.skill_name}") # observability hook
    elif event.type == "text":
        print(event.text, end="")
    elif event.type == "error":
        # M16-style guardrail: surface failures, don't silently fall back.
        raise RuntimeError(f"agent error: {event.message}")

```

```

import { ClaudeAgentClient } from "@anthropic-ai/claude-agent-sdk";

const agent = new ClaudeAgentClient({
  model: "claude-opus-4-7",
  skillDirs: ["./.claude/skills"], // one or many folders
  allowedTools: ["Read", "Write", "Bash"],
});

for await (const event of agent.run("Extract the totals table from invoice.pdf")) {
  switch (event.type) {
    case "skill_loaded":
      console.log(`→ loaded skill: ${event.skillName}`);
      break;
    case "text":
      process.stdout.write(event.text);
      break;
    case "error":
      // M16-style guardrail: surface failures, don't silently fall back.
      throw new Error(`agent error: ${event.message}`);
  }
}

```

What Just Happened?

only the frontmatter (the description) is in the base context.

Skills vs Tools vs MCP vs Subagents — the Honest Picker

Need	Reach for	Why
Single function call (one verb)	Tool (M05)	Schema in, schema out, no instructions needed
Connect to a system (Postgres, GitHub, Jira)	MCP (M07)	One server, many tools, swap clients freely
Procedure with branches + gotchas, used occasionally	Skill	Loaded only when the description matches — cheap in steady state
Parallel specialist with its own context window	Subagent (M14)	Separate context budget; can run concurrently
Capability owned by a different team / org	A2A (above)	Wire format + auth across trust boundaries

Rule of thumb: if you find yourself prepending the same 200-line preamble to a prompt only when a specific kind of file shows up — you wanted a Skill.

Skills are the “just-in-time playbook” capability. The capability further along the timeline — Computer Use — gives agents direct GUI access. Below is what an actual Computer Use loop looks like end-to-end.

Computer Use in Practice: A Working Example

Computer Use is a *pre-built* tool that ships with the Claude API — you don’t define a JSON schema for it, you just enable it and provide an execution sandbox. Claude receives a screenshot, decides on an action (click, type, scroll, key), and your code executes that action and returns the next screenshot. The loop continues until Claude reports it’s done.

Three pre-built tools usually go together for an autonomous Computer Use agent:

- **computer_20250124** — the screenshot/click/type/scroll tool itself.
- **bash_20250124** — lets Claude open URLs, launch apps, restart services.
- **text_editor_20250429** — lets Claude read/write files (e.g., scratch notes for multi-step tasks).

Here is what a Computer Use call looks like end-to-end. The key differences from regular tool use (M05): you set a `betas=["computer-use-2025-01-24"]` header, you pass `display_width_px / display_height_px` so Claude knows the screen geometry, and your loop must execute the action and return a fresh screenshot as the `tool_result`:

```

from anthropic import Anthropic
import base64, subprocess

client = Anthropic()

# Your sandbox primitives. In production these run inside a Docker container
# with a virtual display (Xvfb) so the agent can't touch your real desktop.
def take_screenshot() -> bytes:
    subprocess.run(["scrot", "/tmp/screen.png"], check=True)
    return open("/tmp/screen.png", "rb").read()

def execute_action(action: dict) -> bytes:
    if action["action"] == "screenshot":
        pass
    elif action["action"] == "left_click":
        x, y = action["coordinate"]
        subprocess.run(["xdotool", "mousemove", str(x), str(y), "click", "1"])
    elif action["action"] == "type":
        subprocess.run(["xdotool", "type", "--", action["text"]])
    elif action["action"] == "key":
        subprocess.run(["xdotool", "key", action["text"]])
    # ... scroll, double_click, mouse_move, cursor_position, etc.
    return take_screenshot()

# Initial request: enable the three pre-built tools and give a goal
messages = [{"role": "user", "content": "Open firefox, search 'anthropic pricing', screenshot the result"}]
initial = take_screenshot()

while True:
    response = client.beta.messages.create(
        model="claude-opus-4-7",
        max_tokens=4096,
        tools=[
            {"type": "computer_20250124", "name": "computer",
             "display_width_px": 1280, "display_height_px": 800},
            {"type": "bash_20250124", "name": "bash"},
            {"type": "text_editor_20250429", "name": "str_replace_based_edit_tool"},
        ],
        betas=["computer-use-2025-01-24"],
        messages=messages,
    )

    if response.stop_reason == "end_turn":
        break # Claude decided it's done

    # Append Claude's reply to the conversation
    messages.append({"role": "assistant", "content": response.content})

    # Execute every tool_use block and collect tool_results
    results = []
    for block in response.content:

```

```

        if block.type == "tool_use" and block.name == "computer":
            screenshot = execute_action(block.input)
            results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": [{
                    "type": "image",
                    "source": {"type": "base64", "media_type": "image/png",
                               "data": base64.b64encode(screenshot).decode()}
                }],
            })
        # ... handle bash and text_editor tool_use blocks similarly
    messages.append({"role": "user", "content": results})

```

```

import Anthropic from "@anthropic-ai/sdk";
import { execSync } from "node:child_process";
import { readFileSync } from "node:fs";

const client = new Anthropic();

function takeScreenshot(): Buffer {
    execSync("scrot /tmp/screen.png");
    return readFileSync("/tmp/screen.png");
}

function executeAction(action: any): Buffer {
    if (action.action === "left_click") {
        const [x, y] = action.coordinate;
        execSync(`xdotool mousemove ${x} ${y} click 1`);
    } else if (action.action === "type") {
        execSync(`xdotool type -- ${JSON.stringify(action.text)}`);
    } else if (action.action === "key") {
        execSync(`xdotool key ${action.text}`);
    }
    // ... screenshot, scroll, double_click, mouse_move, cursor_position
    return takeScreenshot();
}

const messages: any[] = [{
    role: "user",
    content: "Open firefox, search 'anthropic pricing', screenshot the result",
}];

while (true) {
    const response = await client.beta.messages.create({
        model: "claude-opus-4-7",
        max_tokens: 4096,
        tools: [
            { type: "computer_20250124", name: "computer",
              display_width_px: 1280, display_height_px: 800 },
            { type: "bash_20250124", name: "bash" },
            { type: "text_editor_20250429", name: "str_replace_based_edit_tool" },

```

```

    ],
    betas: ["computer-use-2025-01-24"],
    messages,
  });

  if (response.stop_reason === "end_turn") break;
  messages.push({ role: "assistant", content: response.content });

  const results: any[] = [];
  for (const block of response.content as any[]) {
    if (block.type === "tool_use" && block.name === "computer") {
      const screenshot = executeAction(block.input);
      results.push({
        type: "tool_result",
        tool_use_id: block.id,
        content: [{
          type: "image",
          source: { type: "base64", media_type: "image/png",
            data: screenshot.toString("base64") },
        }],
      });
    }
  }
  messages.push({ role: "user", content: results });
}

```

⚠ Sandboxing Is Not Optional

The agent above can click and type anywhere on the display. **Never run this against your real desktop.**

Production deployments run the loop inside a disposable Docker container with:

- A virtual display (Xvfb) — no host GUI is touched.
- A locked-down browser/app set — only what the task needs.
- Network egress allow-lists — no exfiltration to arbitrary hosts.
- No host-filesystem mounts — the container is throwaway.
- A wall-clock budget and step cap — prevent runaway loops.

Anthropic publishes a reference Docker image that bundles Xvfb, Firefox, an X11 desktop, and the loop wrapper. Use it as a starting point rather than rolling your own. **Prompt injection from the page being browsed is the #1 risk:** a hostile webpage can include text instructing Claude to leak credentials or click destructive buttons. Pair Computer Use with the input-guardrails patterns from M16 (allow-list of intents, suspicious-action detection, human-in-the-loop confirmation for irreversible actions).

When (and When Not) to Reach for Computer Use

Use it when: the target system has no API, no MCP server, and no scriptable interface (legacy enterprise apps, vendor portals, GUI-only desktop software). Or when you need the same agent to handle apps you can't enumerate ahead of time (general "use this website for me" flows).

Skip it when: a regular API or MCP server exists. Computer Use is slower (each step round-trips a screenshot through vision), more expensive (every screenshot is a full image input), and less reliable (UIs change, rendering differs, modal dialogs surprise the agent). Per-step cost on Opus 4.7: a 1280×800 PNG decodes to roughly 1,500-2,500 vision tokens — a 20-step task can run \$0.20-\$0.50 just on screenshot input. A REST call would cost cents.

The realistic shape of a Computer Use agent in production: a narrow, well-scoped task ("file this expense in Concur," "refresh this tableau dashboard"), inside a locked-down sandbox, with a human-approval step before any irreversible action, and a hard step/time budget. Open-ended "just browse the web for me" agents make impressive demos but are still unreliable enough that most teams keep them out of customer-facing flows.

Why It Matters

Most developers will continue using Claude as a chatbot. You now know how to build agents that use tools (M05-M07), retrieve knowledge (M09-M10), coordinate workflows (M12-M14), and operate in production (M19-M22). Every new capability Claude gains multiplies the value of these skills. When computer use became available, developers who already understood tool use patterns could adopt it in hours. Developers starting from scratch needed weeks. Your foundation is the moat.

Common Misconceptions

"Agents will replace developers." — They amplify, not replace. Someone still defines the problem, sets guardrails, evaluates quality, and handles the edges agents can't. Demand for developers who manage agents is growing.

"Full autonomy is right around the corner." — Current agents excel at well-defined bounded tasks. Open-ended autonomy in high-stakes domains is still unreliable; successful production agents use HITL (M17), not blind autonomy.

"Bigger model = better agent." — Far less than prompt design (M03), tool definitions (M05), guardrails (M16-M17), evaluation (M18). A well-designed small-model agent beats a sloppy big-model one routinely.

More capability means more power — and more power demands more responsibility. The next section covers the ethical and safety principles that separate capable agents from trustworthy ones.

Building Responsibly: Ethics & Human Oversight

Everyday Analogy

A powerful car needs brakes, seatbelts, and airbags. Nobody installs these because they expect to crash — they install them because safety features are what make speed useful. A car without brakes is not fast; it is dangerous. A fast car with good brakes is a tool; a fast car without brakes is a liability.

The pain of skipping safety is catastrophic when it happens, but invisible until then. Every major AI incident (chatbots endorsing harmful advice, agents making unauthorized transactions, automated systems amplifying bias) happened because the builder assumed the happy path was the only path.

Responsible agent development is not a separate discipline from capable agent development — it IS capable agent development. The guardrails from M16-M17, the evaluation from M18, the monitoring from M19-M20 are not optional extras. They are what make your agent trustworthy enough to deploy. The mark of a senior agent developer is knowing when NOT to automate.

Technical Definition

Responsible agent development requires five principles, each of which maps to course modules you have already learned:

- 1. Human-in-the-Loop by Design:** Critical decisions (medical, financial, legal) should require human approval, not just human override. You learned this in M17 — design approval workflows, not just kill switches. The difference: a kill switch assumes the human is watching. An approval gate assumes the human is needed.
- 2. Transparency:** Users should know they are interacting with an AI. They should understand what the agent can and cannot do. In high-stakes domains (healthcare, legal), they should see the reasoning behind decisions. This is not just ethical — it is practical: transparent agents build user trust, and trusted agents get adopted.
- 3. Bias Awareness:** Agents inherit biases from training data and amplify them through automation scale. A biased human loan officer affects dozens of applications per day. A biased agent affects thousands. Regular auditing with diverse test cases (M18) is essential — not once, but continuously.
- 4. Scope Limitation:** Agents should refuse to act outside their defined domain rather than confidently hallucinating. Your system prompt defines what the agent does and does not do. The guardrails from M16 enforce it. An agent that says "I cannot help with that, but here is who can" is infinitely more useful than one that confidently gives wrong advice.
- 5. Fail-Safe Defaults:** When uncertain, defer to humans. When failing, fail open rather than fail closed. What does that mean in practice? "Fail open" means the agent shares what it knows and defers the final decision to a human. "Fail closed" means the agent blocks the user entirely — no information, no action, dead end.

In most cases, fail-open is safer and more useful. If a medical pre-auth agent cannot determine coverage, the right response is: "Here is what I found so far, but I could not confirm coverage for CPT code 27447. Escalating to a clinical reviewer." The wrong response is: "Error: unable to process request."

The circuit breaker pattern from M17 is one implementation of fail-safe defaults. But the principle applies everywhere: uncertainty is not a reason to stop. It is a reason to hand off.

AGENT DECISION SCENARIOS — ESCALATION FRAMEWORK

Query	Decision	Reason
"What's the status of order PO-2024-8847?"	PROCEED	Routine query, within scope, no risk
"Compare loan rates for these two applicants"	FLAG — BIAS	Potential for demographic bias — flag for human review
"Approve this \$50,000 surgical pre-auth"	ESCALATE	High-stakes medical decision — requires physician sign-off

Common Misconceptions

"Responsible AI slows down development." — It does the opposite. Teams that build guardrails and evaluation from the start deploy faster because they spend less time firefighting production failures. The "move fast and break things" philosophy does not work when the thing you break is a medical decision or a financial transaction.

"My agent is just a prototype, so safety doesn't matter yet." — Prototypes become products faster than you expect. Building safety in from the start is dramatically cheaper than retrofitting it later. A 2024 survey found that retrofitting guardrails cost 4× more than building them in during initial development.

"Claude's built-in safety is enough." — Claude has excellent built-in safety, but it cannot know your business rules, your compliance requirements, or your users' specific vulnerabilities. Your guardrails complement Claude's — they do not replace each other.

Responsible building requires staying current with best practices, which evolve as fast as the technology itself. The next section maps the communities, resources, and frameworks that will keep your skills sharp.

Resources: Communities, Papers & Frameworks

Everyday Analogy

Finishing this course is like getting your driver's license. You are qualified to drive — you know the rules, you can handle the vehicle, you have passed the tests. But the best drivers keep learning: from other drivers, from changing road conditions, from new vehicle features, and from their own experience.

The pain of learning in isolation is stagnation. A developer who finishes one course and never reads another blog post, joins a community, or examines a new framework will be using outdated patterns within 6 months. The AI field moves faster than any other area of software engineering.

Here is your map to continued growth: the communities where practitioners share what works, the papers where researchers push boundaries, the frameworks where engineers build tools, and the benchmarks where everyone measures progress.

Not all resources are created equal, and knowing how to evaluate them is a skill in itself. Communities like the Anthropic Developer Forum and AI Engineer Discord give you real-time signal — what practitioners are actually building, what is breaking in production, and what patterns are emerging. This is where you hear about a new technique weeks before it shows up in a blog post.

Research papers and blog posts give you depth. Simon Willison's blog, for example, consistently breaks down new AI capabilities with practical examples and honest assessments of limitations. The Anthropic research publications explain the reasoning behind design decisions in Claude — understanding WHY the model behaves a certain way makes you a dramatically better agent builder.

Open-source frameworks like LangGraph, CrewAI, and the Claude Agent SDK give you building blocks. But be cautious: frameworks change fast, and coupling your entire architecture to one framework can create painful migration later. Use frameworks for orchestration and glue code, but keep your core agent logic framework-agnostic where possible.

THE AGENT DEVELOPER ECOSYSTEM

 Communities

- Anthropic Developer Forum
- AI Engineer community
- LangChain Discord
- LlamaIndex Discord
- Weights & Biases community

 Essential Reading

- Anthropic research publications
- Claude documentation guides
- Simon Willison’s AI blog
- Constitutional AI papers
- RLHF research

The 2026 Layer — What’s New Since the Course Was Recorded

The clusters above are foundational. Three categories have crystallized in 2025–2026 that didn’t exist when most agent courses were written. Track these even if you don’t adopt them yet — they show up in every “state of agents” post for a reason.

 Agentic Memory Frameworks

Beyond “stuff the conversation in a list” (M08) and basic vector RAG (M09). Agentic memory frameworks model memory as a first-class subsystem with reads, writes, evictions, and consolidation — closer to how a human assistant remembers facts about you across sessions.

Project	What it adds	When to reach for it
Letta (formerly MemGPT)	Tiered memory (core, recall, archival) + memory blocks the agent can edit itself	Long-lived personal assistant; agent that learns user preferences
mem0	Drop-in memory layer; LLM-driven fact extraction + relevance scoring	Add memory to an existing agent without redesigning storage
Claude SDK memory blocks	Native, file-backed memory that the agent edits via tool use	Stay on the SDK; lighter than a separate memory service

MCP Ecosystem (Past the Basics)

M07 / CC9 cover what MCP *is*. By 2026 the production stack around MCP looks like this:

Topic	What to read / try	Why it matters
MCP Auth (OAuth 2.1 + PKCE)	MCP authorization spec	Bearer tokens are dead for production; remote MCP servers need real OAuth
MCP UI	mcpui.dev + Anthropic Apps SDK	Tools can return rich UI (cards, forms), not just text; agents that can show, not just tell
MCP Inspector	npx @modelcontextprotocol/inspector	Test-drives a server interactively; standard debugging tool
Sampling-as-protocol	MCP “sampling” capability	Servers can ask the client to call its LLM — nested reasoning without server-side keys
Public server registries	MCP server catalog	Hundreds of vetted servers (GitHub, Slack, Postgres...) — stop building your own first

Agentic Browsers & Workspaces

A different kind of frontier — not protocols but new *surfaces* where users meet agents. Worth tracking because your agent may end up running inside one of these instead of behind a web app.

- **Perplexity Comet** — AI-first browser; the page itself is an agent surface (summarize, ask, automate).
- **Dia** (Browser Company) — same idea, deeper task automation; commands live alongside the address bar.
- **Arc + Arc Max** — mature AI features layered into a conventional browser; pragmatic baseline.
- **Cursor / Windsurf / VS Code agentic modes** — the IDE-as-agent surface; where your developer-facing agents will likely live.

Agentic RAG (covered in depth in M10)

Retrieval where the agent itself decides which corpus to query, how to phrase the query, and when it has enough — the M12 ReAct loop applied to search. Replaces “one-shot top-k” with a multi-step retrieve-read-refine pattern for multi-hop or multi-corpus questions. [See M10 → Agentic RAG](#) for the full pattern with Python + TS code and a decision guide.

Why It Matters

The agent development field is evolving faster than any course can track. In 2024, MCP went from proposal to widespread adoption in under 6 months. In 2025, A2A landed and the agentic-memory market crystallized into Letta + mem0 + SDK-native. Developers who follow even one community (30 minutes per week) stay 6–12 months ahead of developers who learn only from courses and documentation. Your most valuable skill is not what you learned here — it is the ability to evaluate and adopt new tools, patterns, and capabilities as they emerge.

Communities, papers, and frameworks give you a map of the territory. But the framework list above raises an immediate question every new agent developer asks: do I actually need one of these? The next section is the honest answer.

Agent Frameworks — Do You Need One?

Everyday Analogy

Picking an agent framework is like picking a kitchen. A bare pro kitchen with stove, knives, and pans gives you total control — you can cook anything, but you do every step yourself. A meal-kit service gives you pre-portioned ingredients and a recipe card — you cook faster, but only what the kit lets you cook. A microwave dinner is one button — ready in 3 minutes, but you eat exactly what is in the tray.

The pain of skipping straight to the meal-kit is that when something tastes off, you cannot tell whether it is the recipe, the ingredients, or your technique. You did not understand the underlying steps. The pain of the bare kitchen is that you are slow and you keep reinventing the same sauce every Tuesday.

This course deliberately walked you through the bare kitchen first — raw API calls, your own loop, your own tool dispatcher. Now that you understand the underlying steps, you can pick the right kitchen for each meal, including frameworks that hide most of the work.

What This Course Already Taught You

Three approaches, in order of abstraction:

- **Raw API (M05–M15B):** `client.messages.create()` inside a `while` loop. Full control over every turn, every tool call, every retry.
- **Agent SDK (M26):** `@agent.tool`, hooks, sessions — Anthropic's native framework. Adds production patterns on top of the loop you already understand.
- **Spec-driven (M25):** Write a specification and let Claude Code generate the agent for you. Fastest path from idea to working code.

The Framework Landscape

Beyond Anthropic's tooling there is a wider ecosystem. Here is the honest version — what each does, what you gain, what you give up.

AGENT FRAMEWORKS COMPARISON

Framework	What It Does	Pros	Cons
Anthropic Agent SDK	Native Claude agent framework with tools, hooks, sessions	Designed for Claude, maintained by Anthropic, tight integration	Claude-only, newer ecosystem
LangChain	General-purpose LLM framework with chains, agents, tools	Huge ecosystem, many integrations, lots of examples	Heavy abstraction, frequent breaking changes, vendor-agnostic means optimized for none
LangGraph	State machine framework for multi-agent workflows	Visual graph editor, explicit state management, good for complex flows	Steep learning curve, tied to LangChain ecosystem
CrewAI	Multi-agent role-based framework	Easy to define agent “roles”, good for team-of-agents patterns	Less flexibility for custom patterns, abstraction hides the loop
AutoGen	Microsoft's multi-agent conversation framework	Good for agent-to-agent conversations, research-oriented	Complex setup, Microsoft-centric, less production-focused
Haystack	NLP/RAG-focused pipeline framework	Excellent for RAG pipelines, modular components	Less focused on agents, more on retrieval
Long-horizon coding agents Claude Code, Aider, Cline, Codex CLI, Devin	Repository-scoped agents that plan, edit files, run tests, iterate	Off-the-shelf for codebase work; integrates with git, your IDE, your CI	Not a framework you build on top of — you use it; less flexibility for non-code domains
No framework (raw SDK)	Just the Anthropic Python/Node SDK	Maximum control, minimum dependencies, easiest to debug	More code to write, no built-in patterns

Vendor-Native Agent SDKs

The table above mixes *cross-provider frameworks* (LangChain, CrewAI) with *one vendor SDK* (Anthropic). By 2026 each major model vendor ships their own first-party agent SDK — with hooks, sessions, hosted sandboxes, and built-in A2A interop. If you’ve committed to a primary model, the vendor SDK usually beats a cross-provider framework on developer ergonomics, latency, and feature coverage.

VENDOR SDK COMPARISON

Vendor SDK	Primary models	Hosted sandbox?	A2A native?	Languages
Anthropic claude-agent-sdk	Claude Opus/Sonnet/Haiku 4.x	BYO (Bash/Edit tools); pair with Docker/Modal/E2B	Via wrapper; SDK ships A2A adapter	Python, TypeScript
OpenAI openai-agents	GPT-5 / GPT-4.x / o-series	Yes — hosted Code Interpreter sandbox + browser tool	Yes (Responses API + A2A bridge)	Python, TypeScript
Google google-adk (Agent Development Kit)	Gemini 2.x / 3.x	Vertex AI hosted code-exec; isolated VMs	Yes — Google authored A2A; first- class	Python, Java (Go preview)
Microsoft semantic-kernel / AutoGen	Azure OpenAI + any (provider-agnostic)	Azure Container Apps dynamic sessions	Yes — adopted A2A in 2025	Python, C#, Java

⚡ What's New: OpenAI's Hosted Sandbox

The most consequential 2025 add from OpenAI is the **hosted Code Interpreter sandbox** baked into the Agents SDK — an ephemeral, network-isolated Python container that the agent can run code in without you provisioning anything. It's closer to Anthropic's "just enable the tool" ergonomics for Computer Use, but for arbitrary Python: matplotlib plots, pandas analyses, file-uploads-then-process, scientific computing.

The tradeoff: ergonomics win is real (no Docker, no Xvfb, no infra) but you pay per-second of sandbox time on top of LLM tokens, and you give up control of the container image. For sensitive data the hosted sandbox is a non-starter — egress is to OpenAI's VPC; for hobby/research/most app builders it's the right default. *Anthropic's equivalent* is the Computer Use bash/text-editor tools (covered above) + your own Docker; *Google's* is Vertex AI's code-execution tool.

To make the trade-offs concrete, here's the same simple agent — "answer a question using a single search tool" — in both the Claude and OpenAI SDKs. The shapes are remarkably similar; the differences are what we just discussed.

```

# pip install claude-agent-sdk
from claude_agent_sdk import ClaudeAgentClient, AgentOptions, tool

@tool
def search_filings(state: str, name: str) -> list[dict]:
    """Search UCC filings by state and debtor name."""
    return mock_db.query(state=state, name=name)

agent = ClaudeAgentClient(
    options=AgentOptions(
        model="claude-opus-4-7",
        tools=[search_filings],
        allowed_tools=["search_filings"],
    ),
)

async for event in agent.run("Active filings for Acme Corp in CA"):
    if event.type == "text":
        print(event.text, end="")
    elif event.type == "error":
        raise RuntimeError(event.message)

```

```

# pip install openai-agents
from agents import Agent, Runner, function_tool, CodeInterpreterTool

@function_tool
def search_filings(state: str, name: str) -> list[dict]:
    """Search UCC filings by state and debtor name."""
    return mock_db.query(state=state, name=name)

agent = Agent(
    name="filings-agent",
    model="gpt-5",
    tools=[
        search_filings,
        CodeInterpreterTool(), # <-- hosted sandbox, the new piece
    ],
)

result = Runner.run_sync(agent, "Active filings for Acme Corp in CA")
print(result.final_output)
# Anything the agent computes in the sandbox (plots, files) is in result.artifacts

```

```

# pip install google-adk
from google.adk.agents import Agent
from google.adk.tools import FunctionTool, BuiltInCodeExecutionTool

def search_filings(state: str, name: str) -> list[dict]:
    """Search UCC filings by state and debtor name."""
    return mock_db.query(state=state, name=name)

```

```

agent = Agent(
    name="filings-agent",
    model="gemini-3.0-pro",
    tools=[
        FunctionTool(func=search_filings),
        BuiltInCodeExecutionTool(), # <-- Vertex AI hosted code-exec
    ],
)

response = agent.run("Active filings for Acme Corp in CA")
for chunk in response:
    print(chunk.text, end="")

```

Same pattern (model + tools + loop), same author intent. Where they diverge is in the *built-in* tools: OpenAI ships `CodeInterpreterTool` with a hosted Python sandbox, Google ships `BuiltInCodeExecutionTool` backed by Vertex VMs, Anthropic favors a “BYO sandbox + open primitives” design (Bash, Edit, Computer Use). All three speak A2A on the outside — so an orchestrator agent doesn’t care which one is on the other end of a task.

Decision Guide — Picking a Vendor SDK

Anthropic Claude Agent SDK when: you’ve standardized on Claude; you want primitives over magic (BYO sandbox); your sandbox needs are unusual (GPU, GUI, custom OS); production-first patterns (hooks, sessions, subagents).

OpenAI Agents SDK when: you’ve standardized on GPT/o-series; you need an ephemeral Python sandbox *now* without infra work (plots, pandas, scientific compute); you’re fine with OpenAI’s container image and egress rules.

Google ADK when: you’re on Vertex AI / Gemini; A2A is central (Google authored the spec); you need enterprise IAM integration; Java is in your stack.

Microsoft Semantic Kernel / AutoGen when: you’re a C#/Java shop; you want a provider-agnostic SDK that can swap models behind the same agent code; you’re already on Azure.

Cross-vendor: use LangChain/LangGraph only if you genuinely need to swap models routinely and the per-vendor SDK ergonomics aren’t worth the lock-in. For most teams, a vendor SDK + an A2A boundary at trust edges beats a cross-provider abstraction in both performance and maintenance cost.

When to Use What

Decision Guide

Use Anthropic Agent SDK (what this course teaches) when:

- Building Claude-specific agents
- You want hooks for guardrails and sessions for persistence
- You need production-ready patterns without heavy dependencies
- Your team is standardized on Claude

Use LangChain / LangGraph when:

- You need to support multiple LLM providers (Claude + GPT + Gemini)
- You want pre-built integrations (hundreds of tool connectors)
- Your team already uses LangChain

Use raw SDK (no framework) when:

- Learning how agents work (this course's approach)
- Building simple single-purpose agents
- You need maximum control over every API call
- Debugging production issues where frameworks hide the problem

Use CrewAI when:

- Your use case maps cleanly to a “team of specialists” pattern
- You want fast prototyping of multi-agent systems

The Course's Position

Why It Matters

This course deliberately teaches you *without* frameworks first (raw SDK in M05–M15B), then with Anthropic's native framework (Agent SDK in M26), then with spec-driven generation (M25).

If your team uses LangChain, everything you learned transfers — ReAct is ReAct whether you implement it with a `while` loop or LangChain's `AgentExecutor`. The patterns are the same. Only the syntax changes.

The student who understands the raw loop can learn ANY framework in a day. The student who learned LangChain first often cannot debug without it. That is the difference between a developer who happens to use a framework and one who is dependent on it.

Try It Yourself (Optional Exercise)

For students whose teams use LangChain, rebuild the M15B agent with it and compare side-by-side: lines of code, output quality, debuggability, dependency footprint.

```
pip install langchain langchain-anthropic

# Same 3 tools, same mock data, same question as M15B
# But using LangChain's AgentExecutor instead of a raw while loop
# Compare: lines of code, output quality, debuggability, dependencies
```

This is **not** required — it is a “Going Further” exercise for students whose teams use LangChain.

Long-Horizon Coding Agents — A Different Shape Entirely

Most of this course taught you the *one-shot* agent shape: receive a question, loop through tools for seconds-to-a-minute, return an answer. The 2025–2026 wave of **long-horizon coding agents** — Claude Code, Aider, Cline, OpenAI Codex CLI, Devin, Cursor’s background agent — runs on a different clock. They live inside a repository, plan across dozens of files, run tests, fix what they broke, and check work back in — minutes to hours per task. Worth a section because the design tradeoffs flip.

Everyday Analogy

A one-shot agent is a barista — you order, they make it, you leave. A long-horizon coding agent is a contractor — you describe the renovation, they spend days demolishing, framing, plumbing, and fixing what they broke.

One-shot agents (M15B-style) exhaust the context window long before a real codebase task completes. The contractor pattern survives by planning to disk, summarizing turns, and working in checkable steps. You’ll likely *use* one of these (or buy one) rather than build from scratch — but the patterns leak into the agents you do build.

ONE-SHOT AGENT VS LONG-HORIZON CODING AGENT

One-Shot Agent (M15B-style)

- 1 Receive question
- 2 Loop: think → tool → observe
- 3 Stop on end_turn
- 4 Return answer

Horizon: seconds – minutes · **Context:** single window · **State:** in-memory list

Long-Horizon Coding Agent

- 1 Read repo: layout, conventions, CLAUDE.md
- 2 Plan: write spec, decompose tasks
- 3 Loop across files/tests/runs
- 4 Commit, PR, iterate on feedback

Horizon: hours – days · **Context:** disk + vector DB · **State:** file system + DB

If you want to drive one of these from your own code — for nightly maintenance, automated PRs, or red-team scans — the integration surface is usually a CLI plus a config file. Here’s the same task launched two ways: the Claude Agent SDK’s long-running mode (programmatic) and the Claude Code CLI (process-spawned). Pick whichever fits your runtime.

```

import asyncio
from claude_agent_sdk import ClaudeAgentClient, AgentOptions

# A long-horizon coding agent driven by the SDK. The key differences vs
# the M15B-style loop: workspace_dir scopes file access, max_turns is the
# safety budget, and persist_session_id lets us resume if the run is
# interrupted (e.g. nightly cron that picks up where it left off).
async def run_codebase_task(task: str, *, workspace: str, session_id: str):
    agent = ClaudeAgentClient(
        options=AgentOptions(
            model="claude-opus-4-7",
            workspace_dir=workspace,
            allowed_tools=["Read", "Write", "Edit", "Bash", "Grep", "Glob"],
            max_turns=200, # hard ceiling
            persist_session_id=session_id, # resume on next run
            extended_thinking={"budget_tokens": 8000}, # planning compute
        ),
    )

    try:
        async for event in agent.run(task):
            if event.type == "plan_updated":
                print(f"[plan] {event.summary}") # observability
            elif event.type == "tool_call":
                print(f" → {event.tool}({event.args_preview})")
            elif event.type == "text":
                print(event.text, end="")
            elif event.type == "stuck":
                # Long-horizon agents must escalate, not silently spin.
                raise RuntimeError(f"Agent stuck after {event.turn} turns: {event.reason}")
    finally:
        # The agent writes a branch; we leave it for human review (HITL, M17).
        print(f"\nFinished – review branch: claude/{session_id}")

if __name__ == "__main__":
    asyncio.run(run_codebase_task(
        task="Fix all flaky tests in tests/integration/, one PR per root cause.",
        workspace="/srv/repos/payments",
        session_id="nightly-flake-sweep",
    ))

```

```

import { spawn } from "node:child_process";
import { mkdir } from "node:fs/promises";

// Process-spawn pattern: drive the Claude Code CLI from a Node service.
// You get the same long-horizon agent as the SDK example, but you don't
// host the loop – the CLI does. Good for fire-and-forget CI jobs.
async function runCodebaseTask(task: string, workspace: string, sessionId: string) {
    await mkdir(`${workspace}/.claude/sessions`, { recursive: true });

    const child = spawn(

```

```

"claude",
[
  "--print", // non-interactive
  "--cwd", workspace,
  "--allowed-tools", "Read,Write,Edit,Bash,Grep,Glob",
  "--max-turns", "200", // hard ceiling
  "--session-id", sessionId, // resume on next run
  task,
],
{ stdio: ["ignore", "pipe", "pipe"] },
);

child.stdout.on("data", (b) => process.stdout.write(b));
child.stderr.on("data", (b) => process.stderr.write(b)); // plan/tool events

const code: number = await new Promise((res) => child.on("close", res));
if (code !== 0) {
  // Long-horizon agents must escalate, not silently spin.
  throw new Error(`claude exited ${code} for session ${sessionId}`);
}
console.log(`\nFinished – review branch: claude/${sessionId}`);
}

await runCodebaseTask(
  "Fix all flaky tests in tests/integration/, one PR per root cause.",
  "/srv/repos/payments",
  "nightly-flake-sweep",
);

```

When to Buy vs Build

Buy (Claude Code, Aider, Cline, Codex CLI, Devin) for repository-scoped work that looks like a senior engineer's day — bug fixes, test sweeps, refactors, dep upgrades, PR review. The planning + summarization + rollback machinery in these tools is engineer-years of work you won't match on a weekend.

Build when the domain isn't code — multi-hour data-pipeline triage, legal-review on 400-page filings, clinical chart review. Patterns transfer (plan-to-disk, summarize-and-evict, escalate-when-stuck) but no shipping tool covers your domain.

Compose: drive a long-horizon coding agent *from* your domain agent — your insurance-claims agent decides “needs a rule-engine fix” and spawns Claude Code to ship the patch.

Gotchas Specific to Long-Horizon Agents

- **Context bloat is the silent killer** — even 1M-token windows die if every turn is preserved; summarize completed steps, evict raw tool output.
- **Sandbox FS + network** — disposable container + egress allow-list, same threat model as Computer Use.
- **No auto-merge without an evaluator** — plug M18 evaluation in front of any auto-merge; code has to compile, pass tests, not regress prod.

You now know the full landscape: what this course taught you, what other frameworks exist, and when to reach for each one — including the long-horizon agents you'll likely buy rather than build. Knowledge without action is trivia — the next section gives you a concrete plan to turn everything you have learned into real projects, real deployments, and real impact.

Your Personal Agent Development Roadmap

Everyday Analogy

A roadmap is not a rigid schedule — it is a compass bearing. Hikers heading north may detour around a swamp, climb over a ridge, or follow a stream, but they keep checking the compass. The specific path changes with the terrain; the direction stays constant.

The pain of not having a direction is drift. Developers who finish a course with enthusiasm but no plan typically build nothing in the following month. They meant to, they planned to, but without concrete milestones, "I'll start this weekend" becomes "I'll start next month" becomes "I should retake the course."

Your compass bearing is: build more capable, more reliable agents. The milestones below are checkpoints, not deadlines. If you hit them faster, great. If life intervenes and you need longer, that is fine too. The point is having a direction to return to.

Technical Definition

A structured post-course development plan with five milestones. Each milestone builds on the previous one, and each can be measured with the rubric from M23:

Weeks 1–2 — Complete Your Capstone: Finish your chosen capstone project from M23. Run the evaluation harness. Score yourself with the rubric. This is not optional — an unfinished capstone is the difference between "I learned about agents" and "I can build agents."

Month 1 — First Production Deploy: Deploy your capstone agent to production, even at small scale. A personal deployment (your own API key, your own server) counts. The goal is to experience the operational reality: latency, cost, monitoring, and real user behavior that test scenarios do not cover.

Months 2–3 — Novel Agent Project: Build an agent for a problem you personally care about. This is where real learning happens because YOU define the requirements. Nobody hands you a brief — you discover the edge cases, choose the architecture, and own the outcome.

Months 3–6 — Community Contribution: Open-source a tool, write about what you learned, or help others in communities. Teaching solidifies understanding. A blog post about a tricky deployment issue helps more people than a perfect but private codebase.

Ongoing — Deep Specialization: Follow one research thread deeply: multi-agent systems, evaluation methods, safety/alignment, or domain-specific agents. Breadth got you here; depth keeps you growing. Set a concrete 90-day goal and track progress monthly.

YOUR 90-DAY AGENT DEVELOPER ROADMAP

★ YOU ARE HERE — Module 24 of 24 complete

- 1 Complete Capstone Project — Weeks 1–2**
Build the capstone end-to-end; document what you learned.
- 2 First Production Deploy — Month 1**
Ship one agent to real users; instrument with tracing.
- 3 Novel Agent Project — Months 2–3**
Tackle a problem you care about; go beyond course examples.
- 4 Community Contribution — Months 3–6**
Publish a library, blog post, or open-source agent.
- 5 Deep Specialization — Ongoing**
Pick one domain (RAG, multi-agent, eval) and go deep.

Why It Matters

Developers who set a concrete 90-day goal are 3× more likely to ship a project than those who have a vague "I should build something." The goal does not need to be ambitious — "deploy a document analysis agent that handles 10 queries/day for my team" is better than "build a revolutionary AI product." Small, specific, and shipped beats large, vague, and unfinished every time.

New Project Template

When you start your next agent project, use this template. It pre-configures the best practices from every track in the course: structured logging, guardrails, cost tracking, and the agentic loop. Everything you need, nothing you do not.

Let's walk through what this template gives you, section by section.

The first thing you will notice is the structured JSON logging from M19. Every log line is machine-parseable, which means your monitoring tools — Datadog, CloudWatch, or even a simple `jq` command in your terminal — can filter and search logs without regex gymnastics. This is the kind of decision that feels trivial on day one but saves you hours on day thirty when something breaks at 2 AM and you need to find the failing request.

Next, the cost tracking section. It accumulates token usage per session, so you always know how much a conversation is costing before it spirals. Why does this matter? Because a runaway agentic loop that calls tools 50 times can burn through your API budget in minutes. The `track_cost` function gives you visibility so you can set alerts or hard caps.

Now the interesting part: the agentic loop in `run_agent`. This is the exact pattern from M12. Here is the flow: send a message to Claude, check `stop_reason`, handle tool calls if the model asks for them, then loop. The loop ends when Claude says "end_turn" or you hit the safety cap. That safety cap (`MAX_ITERATIONS = 10`) is a backstop, not the primary control. Your agent should terminate naturally via `stop_reason` in almost every case. If you are regularly hitting the cap, that is a sign your tools or prompts need debugging, not that the cap needs raising.

Finally, the input guardrails. They are deliberately simple here — just length validation and empty-input checks. In a real project, you would add domain-specific validation from M16: profanity filters, PII detection, prompt injection checks. The template gives you the hook point; you fill in the rules for your domain.

```

# new_agent_project/agent.py
# Production-ready agent template with best practices from M01-M22.
# Includes: structured logging, guardrails, cost tracking, and
# the agentic loop pattern from M12.

import anthropic
import json
import logging
import time
from datetime import datetime

# --- Structured Logging (M19) ---
# JSON-formatted logs with request IDs for tracing.
logging.basicConfig(
    level=logging.INFO,
    format='{ "time": "%(asctime)s", "level": "%(levelname)s", "msg": "%(message)s" }',
)
logger = logging.getLogger(__name__)

# --- Configuration ---
MODEL = "claude-sonnet-4-6"
MAX_TOKENS = 1024
MAX_ITERATIONS = 10 # Safety net (M12: stop_reason is primary)

# --- Cost Tracking (M22) ---
COST_PER_1K = {"input": 0.003, "output": 0.015}
session_cost = {"input_tokens": 0, "output_tokens": 0}

def track_cost(response):
    """Accumulate token usage for cost monitoring."""
    usage = response.usage
    session_cost["input_tokens"] += usage.input_tokens
    session_cost["output_tokens"] += usage.output_tokens
    cost = (
        usage.input_tokens / 1000 * COST_PER_1K["input"]
        + usage.output_tokens / 1000 * COST_PER_1K["output"]
    )
    logger.info(f"call_cost=${cost:.4f} total_in={session_cost['input_tokens']} total_out={session_cost['output_tokens']}")
    return cost

# --- Input Guardrails (M16) ---
MAX_INPUT_LENGTH = 4000

def validate_input(user_message: str) -> str | None:
    """Return an error message if input is invalid, None if OK."""

```

```

    if not user_message or not user_message.strip():
        return "Please provide a message."
    if len(user_message) > MAX_INPUT_LENGTH:
        return f"Message too long ({len(user_message)} chars, max {MAX_INPUT_LENGTH})."
    return None

# --- Tool Definitions ---
# Replace these with your actual tools.
TOOLS = [
    {
        "name": "example_tool",
        "description": "Replace this with your tool description.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "The input query",
                }
            },
            "required": ["query"],
        },
    },
]

def handle_tool_call(name: str, input_data: dict) -> dict:
    """Dispatch tool calls to handlers. Add your tools here."""
    if name == "example_tool":
        return {"result": f"Tool result for: {input_data.get('query', '')}"}
    return {"is_error": True, "error_category": "unknown_tool", "context": name}

# --- System Prompt ---
SYSTEM_PROMPT = """You are a helpful assistant. Replace this with your
agent's specific role, capabilities, and constraints.

Rules:
- Only answer questions within your defined scope.
- If unsure, say so rather than guessing.
- Never reveal internal system details.
"""

def run_agent(user_message: str) -> tuple[str, list[str], float]:
    """
    Run the agent for a single user message.
    Returns (response_text, tools_called, total_cost).
    """
    # Input validation (M16)
    error = validate_input(user_message)

```



```

if error:
    return error, [], 0.0

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY env var
messages = [{"role": "user", "content": user_message}]
tools_called = []
total_cost = 0.0
request_id = datetime.now().strftime("%Y%m%d%H%M%S%f")

logger.info(f"request_id={request_id} input_length={len(user_message)}")

for iteration in range(MAX_ITERATIONS):
    try:
        response = client.messages.create(
            model=MODEL,
            max_tokens=MAX_TOKENS,
            system=SYSTEM_PROMPT,
            tools=TOOLS,
            messages=messages,
        )
        total_cost += track_cost(response)
    except anthropic.APIError as e:
        logger.error(f"request_id={request_id} api_error={e}")
        return f"I encountered a technical issue. Please try again.", tools_called, total_cost

# Primary stop condition (M12): check stop_reason
if response.stop_reason == "tool_use":
    tool_results = []
    for block in response.content:
        if block.type == "tool_use":
            tools_called.append(block.name)
            logger.info(f"request_id={request_id} tool={block.name} iteration={iteration}")
            result = handle_tool_call(block.name, block.input)
            tool_results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": json.dumps(result),
            })
    messages.append({"role": "assistant", "content": response.content})
    messages.append({"role": "user", "content": tool_results})

elif response.stop_reason == "end_turn":
    text = " ".join(
        b.text for b in response.content if hasattr(b, "text")
    )
    logger.info(f"request_id={request_id} done iterations={iteration+1} tools={tools_called} co
    return text, tools_called, total_cost

logger.warning(f"request_id={request_id} hit_max_iterations={MAX_ITERATIONS}")
return "I was unable to complete your request.", tools_called, total_cost

```

```

if __name__ == "__main__":
    query = input("You: ")
    response, tools, cost = run_agent(query)
    print(f"\nAgent: {response}")
    print(f"Tools: {tools} | Cost: ${cost:.4f}")

// new_agent_project/agent.ts
// Production-ready agent template with best practices from M01-M22.
// Includes: structured logging, guardrails, cost tracking, and
// the agentic loop pattern from M12.

import Anthropic from "@anthropic-ai/sdk";

// --- Configuration ---
const MODEL = "claude-sonnet-4-6";
const MAX_TOKENS = 1024;
const MAX_ITERATIONS = 10; // Safety net (M12: stop_reason is primary)

// --- Cost Tracking (M22) ---
const COST_PER_1K = { input: 0.003, output: 0.015 };
const sessionCost = { inputTokens: 0, outputTokens: 0 };

function trackCost(response: Anthropic.Message): number {
    const { input_tokens, output_tokens } = response.usage;
    sessionCost.inputTokens += input_tokens;
    sessionCost.outputTokens += output_tokens;
    const cost =
        (input_tokens / 1000) * COST_PER_1K.input +
        (output_tokens / 1000) * COST_PER_1K.output;
    console.log(JSON.stringify({
        time: new Date().toISOString(), level: "INFO",
        msg: `call_cost=${cost.toFixed(4)} total_in=${sessionCost.inputTokens} total_out=${sessionCost.outputTokens}`
    }));
    return cost;
}

// --- Structured Logging (M19) ---
function log(level: string, msg: string, data: Record<string, unknown> = {}) {
    console.log(JSON.stringify({ time: new Date().toISOString(), level, msg, ...data }));
}

// --- Input Guardrails (M16) ---
const MAX_INPUT_LENGTH = 4000;

function validateInput(message: string): string | null {
    if (!message || !message.trim()) return "Please provide a message.";
    if (message.length > MAX_INPUT_LENGTH)
        return `Message too long (${message.length} chars, max ${MAX_INPUT_LENGTH}).`;
    return null;
}

```

```
// --- Tool Definitions ---
const TOOLS: Anthropic.Tool[] = [
  {
    name: "example_tool",
    description: "Replace this with your tool description.",
    input_schema: {
      type: "object" as const,
      properties: {
        query: { type: "string", description: "The input query" },
      },
      required: ["query"],
    },
  },
];

function handleToolCall(name: string, input: Record<string, string>): unknown {
  if (name === "example_tool") {
    return { result: `Tool result for: ${input.query ?? ""}` };
  }
  return { isError: true, errorCategory: "unknown_tool", context: name };
}

// --- System Prompt ---
const SYSTEM_PROMPT = `You are a helpful assistant. Replace this with your
agent's specific role, capabilities, and constraints.

Rules:
- Only answer questions within your defined scope.
- If unsure, say so rather than guessing.
- Never reveal internal system details.`;

export async function runAgent(
  userMessage: string,
): Promise<[string, string[], number]> {
  // Input validation (M16)
  const error = validateInput(userMessage);
  if (error) return [error, [], 0];

  const client = new Anthropic(); // reads ANTHROPIC_API_KEY env var
  const messages: Anthropic.MessageParam[] = [
    { role: "user", content: userMessage },
  ];
  const toolsCalled: string[] = [];
  let totalCost = 0;
  const requestId = Date.now().toString(36);

  log("INFO", "request_start", { requestId, inputLength: userMessage.length });

  for (let i = 0; i < MAX_ITERATIONS; i++) {
    let response: Anthropic.Message;
    try {
```

```

    response = await client.messages.create({
      model: MODEL,
      max_tokens: MAX_TOKENS,
      system: SYSTEM_PROMPT,
      tools: TOOLS,
      messages,
    });
    totalCost += trackCost(response);
  } catch (err) {
    log("ERROR", "api_error", { requestId, error: String(err) });
    return ["I encountered a technical issue. Please try again.", toolsCalled, totalCost];
  }

  if (response.stop_reason === "tool_use") {
    const toolResults: Anthropic.ToolResultBlockParam[] = [];
    for (const block of response.content) {
      if (block.type === "tool_use") {
        toolsCalled.push(block.name);
        log("INFO", "tool_call", { requestId, tool: block.name, iteration: i });
        const result = handleToolCall(block.name, block.input as Record<string, string>);
        toolResults.push({
          type: "tool_result",
          tool_use_id: block.id,
          content: JSON.stringify(result),
        });
      }
    }
    messages.push({ role: "assistant", content: response.content });
    messages.push({ role: "user", content: toolResults });

  } else if (response.stop_reason === "end_turn") {
    const text = response.content
      .filter((b): b is Anthropic.TextBlock => b.type === "text")
      .map((b) => b.text)
      .join(" ");
    log("INFO", "request_done", { requestId, iterations: i + 1, tools: toolsCalled, cost: totalCost });
    return [text, toolsCalled, totalCost];
  }
}

log("WARN", "max_iterations", { requestId, max: MAX_ITERATIONS });
return ["I was unable to complete your request.", toolsCalled, totalCost];
}

```

What Just Happened?

Hands-On Exercise

What You'll Build: A personalized agent project using the production template, plus a written 90-day development plan.

Time Estimate: 45–60 minutes

Prerequisites: Python 3.9+ or Node.js 18+ installed, ANTHROPIC_API_KEY set in your environment, a text editor.

Step 1: Set Up Your Project From the Template

Copy the template from the previous section into a new project directory. This gives you the production-ready scaffolding with logging, guardrails, and cost tracking already wired in.

```
mkdir my-agent-project && cd my-agent-project
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install anthropic
export ANTHROPIC_API_KEY=your-key-here # Windows: set ANTHROPIC_API_KEY=your-key-here

# Create agent.py and paste the Python template from the section above
```

```
mkdir my-agent-project && cd my-agent-project
npm init -y
npm install @anthropic-ai/sdk
export ANTHROPIC_API_KEY=your-key-here # Windows: set ANTHROPIC_API_KEY=your-key-here

# Create agent.ts and paste the Node.js template from the section above
```

Expected Output (after running the agent)

```
You: Hello, what can you do?
{"time":"2026-04-02...", "level":"INFO", "msg": "request_id=... input_length=24"}
{"time":"2026-04-02...", "level":"INFO", "msg": "call_cost=$0.0012 total_in=156 total_out=42"}
{"time":"2026-04-02...", "level":"INFO", "msg": "request_id=... done iterations=1 tools=[] cost=$0.0012"}

Agent: I'm a helpful assistant. I can answer questions within my defined scope...
Tools: [] | Cost: $0.0012
```

Checkpoint

Step 2: Replace the Example Tool

What & Why: Replace `example_tool` with something useful to you. A tool that returns real (or mock) data teaches you the full loop: Claude decides to call the tool, your code executes it, and Claude incorporates the result into its response.

Here are three ideas depending on your interest:

- A **weather lookup tool** that returns mock weather data for a city
- A **file search tool** that lists files matching a pattern in a directory
- A **calculation tool** that performs math operations the agent can call

Update three things in your code: (1) the `TOOLS` array with your new tool definition, (2) the `handle_tool_call` function with your tool logic, and (3) the `SYSTEM_PROMPT` to describe your agent's new specialty.

Run Command (Python)

```
python agent.py
# Then type a query that should trigger your tool, e.g.:
You: What's the weather in San Francisco?
```

Expected Output

```
{"time":"2026-04-02...", "level":"INFO", "msg":"request_id=... input_length=39"}
{"time":"2026-04-02...", "level":"INFO", "msg":"request_id=... tool=weather_lookup
iteration=0"}
{"time":"2026-04-02...", "level":"INFO", "msg":"call_cost=$0.0018 total_in=210 total_out=58"}
{"time":"2026-04-02...", "level":"INFO", "msg":"request_id=... done iterations=2 tools=
['weather_lookup'] cost=$0.0035"}
```

Agent: Based on the weather data, San Francisco is currently 62°F and partly cloudy...
Tools: ['weather_lookup'] | Cost: \$0.0035

✅ Checkpoint

Troubleshooting:

- If the agent does not call the tool, your tool description is probably too vague. Make it specific: "Returns current weather data for a given city name" not "Does weather stuff."

- If you see `unknown_tool` in the logs, your `handle_tool_call` function does not match the tool name in your `TOOLS` definition. Check the spelling.
- If you see `ValidationError`, your tool's `input_schema` does not match what Claude is sending. Add `print(input_data)` in your handler to see the actual input.

Step 3: Write Your 90-Day Goal

What & Why: Create a file called `ROADMAP.md` in your project directory. Research shows developers who write down concrete goals are 3× more likely to ship. This step takes 15 minutes but is the difference between "I'll build something eventually" and "I'm deploying a document Q&A agent by May 15."

Write your personal 90-day development plan using this template:

```
# My 90-Day Agent Development Roadmap

## Goal (be specific)
Deploy a [type] agent that handles [metric] for [audience].
Example: "Deploy a document analysis agent that handles 100 queries/day for my team."

## Weeks 1-2: Finish Capstone
- [ ] Which capstone? ____
- [ ] Run evaluation harness
- [ ] Score with rubric: Functionality __ / Code Quality __ / Prompts __ / Safety __ / Observability __

## Month 1: First Production Deploy
- [ ] Deploy target: [local / cloud / team server]
- [ ] Monitoring setup: [structured logs / dashboard]
- [ ] First real user test date: ____

## Months 2-3: Novel Project
- [ ] Problem I want to solve: ____
- [ ] Why I care about this: ____
- [ ] Architecture sketch: [single agent / multi-agent / RAG]

## Weakest Dimension
My weakest rubric dimension is: ____
Action to strengthen it: ____

## Community
- [ ] Join: [forum/Discord name]
- [ ] Share what I built by: [date]
```

Checkpoint

Step 4: Join a Community and Introduce Yourself

What & Why: Pick one community from the resources list (Anthropic Developer Forum, AI Engineer community, or a framework-specific Discord). Create an account, introduce yourself, and share one thing you built or learned in this course. This step takes 10 minutes but connects you to the ecosystem that will keep your skills current.

✅ Checkpoint

Step 5: Reflect on Your Weakest Dimension

What & Why: Look at the five rubric dimensions from M23: Functionality, Code Quality, Prompts, Safety, and Observability. Which one is your weakest? Write a specific action in your `ROADMAP.md` to strengthen it. For example: "My weakest dimension is Observability. Action: add structured logging and a cost dashboard to my capstone project by next Friday."

✅ Checkpoint

Final Verification

You should now have:

- A working agent project with a custom tool (test it: `python agent.py`)
- A `ROADMAP.md` with specific, dated goals
- An account on at least one agent developer community

🎉 Congratulations

Stretch goal (OPTIONAL): Write a blog post or tutorial about something you learned in this course. Teaching is the fastest way to solidify understanding.

Stretch goal (OPTIONAL): Contribute to an open-source agent framework. Even a documentation fix or bug report counts as a meaningful contribution to the ecosystem.

Knowledge Check

Q1: What problem does the Model Context Protocol (MCP) solve?

- ☐ A It makes Claude respond faster by compressing prompts
- ☐ B It replaces the need for API keys when using Claude
- ☐ C It standardizes how AI models connect to tools and data sources, eliminating custom integration code
- ☐ D It provides a database for storing agent conversation history

Q2: How does Extended Thinking differ from chain-of-thought prompting?

- ☐ A They are the same thing — both instruct Claude to "think step by step"
- ☐ B Extended thinking is model-native reasoning with dedicated compute and separate billing, not a prompt technique
- ☐ C Extended thinking only works on math problems, while chain-of-thought works on any task
- ☐ D Chain-of-thought is more powerful because the user controls the reasoning process

Q3: An agent is asked to approve a \$50,000 surgical pre-authorization. What is the correct design pattern?

- ☐ A Let the agent decide autonomously — it has been trained on medical data
- ☐ B Refuse the request entirely — agents should never handle medical decisions
- ☐ C Add a disclaimer to the agent's response and let it proceed
- ☐ D The agent analyzes the case and presents its recommendation, but a human clinical reviewer must approve before execution

Q4: Which resource type is most useful for staying current on rapidly evolving agent capabilities?

- ☐ A Communities and practitioner blogs — they cover new developments within days
- ☐ B Textbooks — they provide the most thorough and accurate coverage
- ☐ C Academic papers — they are peer-reviewed and always correct
- ☐ D Video courses — they are the easiest way to learn new concepts

Q5: What is the single most important skill for long-term success as an agent developer?

- ☐ A Mastering Python and TypeScript so you can implement any pattern
- ☐ B Memorizing all Claude API parameters and model specifications
- ☐ C The ability to evaluate and adopt new tools, patterns, and capabilities as they emerge
- ☐ D Building the largest possible agent with the most tools and features

Q6: A travel-booking agent built by your team needs to call a flight-pricing agent owned by an external vendor. Which protocol is the right fit?

- ☐ A MCP — expose the vendor’s pricing engine as an MCP server
- ☐ B An in-process subagent — spawn the vendor agent as a thread
- ☐ C A2A — fetch the vendor’s Agent Card and post tasks to its endpoint
- ☐ D A custom REST API negotiated bilaterally with the vendor

Q7: You find yourself prepending the same 150-line preamble to a prompt only when the user uploads a CSV. What feature is designed for exactly this?

- ☐ A Add it as a system prompt section — always paid for, always present
- ☐ B A Skill — the description is loaded into context, but the procedure body is only loaded when the agent decides the task matches
- ☐ C A new MCP server — expose CSV parsing as a tool
- ☐ D A subagent — spin one up per CSV upload

Module Summary — Course Complete

You have reached the end of the course curriculum. Here is what you explored in this final module:

- **Agent-to-Agent Protocols:** MCP for agent ↔ tool, A2A for agent ↔ agent. Agent Cards, task lifecycles, and a working publish/consume example. AGNTCY and ANP positioned in the wider landscape.
- **Evolving Capabilities:** Computer use, extended thinking, larger context windows, and **Skills** — capability bundles that page in just-in-time instructions. The decision table for tools vs MCP vs Skills vs subagents vs A2A.
- **Responsible Building:** Human-in-the-loop design, transparency, bias awareness, scope limitation, and fail-safe defaults — the principles that make capable agents trustworthy.

- **Resources & the 2026 Layer:** Communities, papers, benchmarks — plus agentic memory frameworks (Letta, mem0), the production MCP stack (auth, UI, Inspector, registries), and agentic browsers (Comet, Dia, Arc).
- **Frameworks & Long-Horizon Coding Agents:** When to use SDK vs LangChain vs CrewAI — and the buy-vs-build calculus for Claude Code, Aider, Cline, Codex CLI, Devin.
- **Your Roadmap:** A concrete 90-day plan from capstone completion through production deployment to community contribution.

You started this course knowing basic programming. You now know how to build, deploy, secure, monitor, and optimize production AI agents. The foundation is laid. What you build next is up to you.

If you are pursuing the Claude Certified Architect certification, continue to [Track 9 \(M25–M27\)](#) for certification-specific content.

References & Resources

Anthropic & Core Docs

- [Claude Tool Use Documentation](#) — Official guide to function calling
- [Computer Use Documentation](#) — Guide to Claude's computer use capability
- [Extended Thinking Documentation](#) — Guide to using extended thinking
- [Claude Skills Documentation](#) — SKILL.md format and skill discovery
- [Claude Agent SDK](#) — Python and Node SDKs for production agents
- [Anthropic Cookbook](#) — Production-ready code examples
- [Claude Model Overview](#) — Current models, capabilities, and pricing

Agent Interop Standards

- [Model Context Protocol](#) — MCP specification and public server directory
- [MCP Authorization Spec](#) — OAuth 2.1 + PKCE for remote MCP servers
- [MCP Inspector](#) — Interactive debugger for MCP servers
- [MCP UI](#) — Rich UI payloads from MCP tool results
- [A2A Specification](#) — Agent2Agent Protocol (Linux Foundation)
- [A2A Reference Implementations](#) — Sample agents in Python and TypeScript
- [AGNTCY \(Internet of Agents\)](#) — Cisco-led open agent stack: identity, discovery, observability
- [Agent Network Protocol \(ANP\)](#) — Decentralized DID-based agent network spec

Agentic Memory Frameworks

- [Letta](#) (formerly MemGPT) — Tiered memory + self-editable memory blocks
- [mem0](#) — Drop-in memory layer with LLM-driven fact extraction
- [MemGPT Original Paper](#) — The OS-inspired memory model that started this category

Long-Horizon Coding Agents

- [Claude Code](#) — Anthropic's official long-horizon coding agent (this course covers it in M25–M26)
- [Aider](#) — Open-source pair-programming agent with git integration
- [Cline](#) — VS Code long-horizon agent
- [OpenAI Codex CLI](#) — Cross-model coding agent